

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#03)

Apply Behavioral-level Verilog to Design and Simulate Combinational Modules

Student Name: _____ **Jawad Ahmed** _____ Roll No: _____ **24i-6570** _____ Section: **A** _____

Date performed: _____ **18, 08, 2026** _____

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____ **Lab Instructors: Dr. Nasir Ali Shah,**
Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:**Description:**

This Verilog code implements a 4-bit behavioral adder/subtractor module that performs addition when mode is 0 and subtraction when mode is 1. The circuit outputs a 5-bit result to manage potential carries during arithmetic operations. A dedicated testbench evaluates addition and subtraction test cases using 50 ns delays to verify functionality.

Code:

```
module adder_subtractor_4bit (
    input [3:0] A,
    input [3:0] B,
    input mode,
    output reg [4:0] result
);
    always @(*)
begin
    if (mode == 1'b0)
        result = A + B;
    else
        result = A - B;
    end
endmodule

module tb_adder_subtractor_4bit;
    reg [3:0] A, B;
    reg mode;
    wire [4:0] result;

    adder_subtractor_4bit uut(A, B, mode, result);

    initial
begin
        mode = 0; A = 4'b0101; B = 4'b0011; #50;
        mode = 0; A = 4'b1111; B = 4'b0001; #50;

        mode = 1; A = 4'b0101; B = 4'b0011; #50;
        mode = 1; A = 4'b1000; B = 4'b0010; #50;
    end
end
```

endmodule

Output:

	/tb_adder_subtractor_4bit/A	0101	0101	1111	0101	1000
	/tb_adder_subtractor_4bit/B	0011	0011	0001	0011	0010
	/tb_adder_subtractor_4bit/mode	0				
	/tb_adder_subtractor_4bit/result	01000	01...	10...	00...	00110

Task 2:

Description:

This Verilog code implements a custom 2-to-1 multiplexer that selects between two logic operations based on the control signal select. When select is 0, output Y computes the AND of inputs A and B, and when set to 1, it computes their OR. The testbench tests all eight possible input combinations using 50 ns delays to verify.

Code:

```
module custom_mux_2x1 (
    input A,
    input B,
    input select,
    output reg Y
);
    always @(*) begin
        if (select == 1'b0)
            Y = A & B;
        else
            Y = A | B;
        end
    endmodule

module t_custom_mux_2x1;
    reg A, B, select;
    wire Y;
```

```
custom_mux_2x1 uut (A, B, select, Y);
```

```
initial
```

```
begin
```

```
    select = 0; A = 0; B = 0; #50;
```

```
    select = 0; A = 0; B = 1; #50;
```

```
    select = 0; A = 1; B = 0; #50;
```

```
    select = 0; A = 1; B = 1; #50;
```

```
    select = 1; A = 0; B = 0; #50;
```

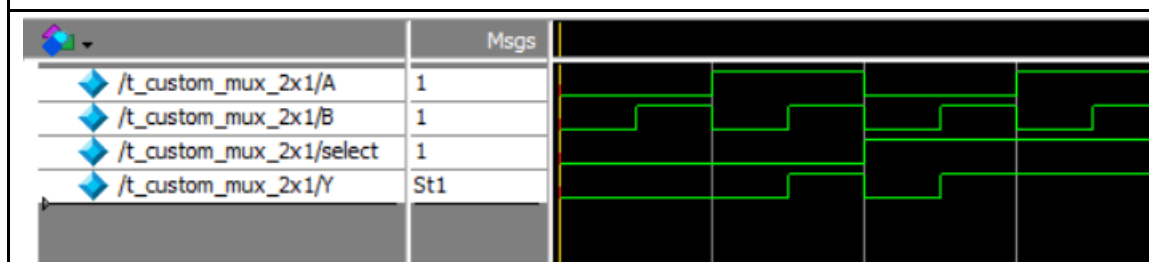
```
    select = 1; A = 0; B = 1; #50;
```

```
    select = 1; A = 1; B = 0; #50;
```

```
    select = 1; A = 1; B = 1; #50;
```

```
end
```

```
endmodule
```

Output:**Task 3:****Description:**

This Verilog code implements a 2-bit magnitude comparator that compares two 2-bit binary numbers, A and B. It uses exactly one of three outputs: A_eq_B, A_gt_B, or A_lt_B—depending on whether A is equal to, greater than, or less than B. The testbench applies 50 ns delays across test cases covering all three comparison outcomes to verify the circuit's behavioral operation.

Code:

```
module comparator_2bit (  
    input [1:0] A,  
    input [1:0] B,  
    output reg A_eq_B,
```

```
output reg A_gt_B,
output reg A_lt_B
);
always @(*) begin
    A_eq_B = 1'b0;
    A_gt_B = 1'b0;
    A_lt_B = 1'b0;

    if (A == B)
        A_eq_B = 1'b1;
    else if (A > B)
        A_gt_B = 1'b1;
    else
        A_lt_B = 1'b1;
end
endmodule

module tb_comparator_2bit;
    reg [1:0] A, B;
    wire A_eq_B, A_gt_B, A_lt_B;

    comparator_2bit uut (A, B, A_eq_B, A_gt_B, A_lt_B);

    initial begin
        A = 2'b00; B = 2'b00; #50;
        A = 2'b11; B = 2'b11; #50;

        A = 2'b10; B = 2'b01; #50;
        A = 2'b11; B = 2'b00; #50;

        A = 2'b00; B = 2'b01; #50;
        A = 2'b01; B = 2'b10; #50;

    end
endmodule
```

Output:

	Msgs	
+ /tb_comparator_2bit/A	01	00 11 10 11 00 01
+ /tb_comparator_2bit/B	10	00 11 01 00 01 10
/tb_comparator_2bit/A_eq_B	St0	
/tb_comparator_2bit/A_gt_B	St0	
/tb_comparator_2bit/A_lt_B	St1	